

Aerial cactus classification

Abdessamed Qchohi
EURECOM

Abdessamed.Qchohi@eurecom.fr

Francesco Giannuzzo
EURECOM

Francesco.Giannuzzo@eurecom.fr

Alessio Giuffrida
EURECOM

Alessio.Giuffrida@eurecom.fr

Matteo Ghia
EURECOM

Matteo.Ghia@eurecom.fr

Abstract—This project tackles the problem of aerial cactus identification, which is based on a competition organized as part of an environmental project in Mexico. Given a dataset of low resolution images, the objective is to correctly label an image as containing a cactus or not. We firstly used various machine learning models, and then we experimented with a transfer learning approach, using pretrained CNN architectures such as ResNet-50 and VGG-16 with a classification head trained on top. We also tried to employ a Visual Question Answering (VQA) model to assign labels to the unlabeled test set.

I. INTRODUCTION

The purpose of this work is to find a model that is able to identify the presence of a cactus in an image. The dataset used was provided by Mexican researchers that started the VIGIA (Vigilancia Autónoma de Reservas de la Biósfera) project [1], who wanted to build a system for autonomous surveillance of protected areas, and needed a system able to recognize the vegetation inside the protected areas.

II. DATASET ANALYSIS

The training dataset contains 17500 images that represent aerial photos of the vegetation. All the images are of size 32×32 and belong to 2 classes: a *csv* file is provided with the id of the image and a label, which takes value 0 if the image does not contain a cactus and 1 otherwise. The first step was analysing the distribution of the labels: Fig. 1 shows that the dataset is highly unbalanced, with the images with a cactus being more than three times as many as the images without.

III. DATA PREPROCESSING AND PREPARATION

Depending on the model used, different preprocessing and feature extraction techniques were applied.

A. Data augmentation

To address the problem of class imbalance, we performed data augmentation: we applied some transformations to the images of the minority class (no cactus labeled as 0), such as random flip with probability 0.5 and rotation with an angle between 0° and 45° chosen from a uniform distribution. This was done to increase the number of images with label 0, so that in the end the number of images of the two classes is balanced. We ended up with a dataset of 26228 images, which served as the starting point for all the models.



Fig. 1. Classes distribution in the dataset

B. Transformations and normalization

For the experiments performed using transfer learning with the ResNet [2] and VGG [3] architectures (which were trained on the ImageNet dataset [4]) we resized all images to 256×256 pixels and then we performed a center crop to 224×224 to match the size of images in ImageNet. Subsequently, we normalized the images using the mean and standard deviation of ImageNet. Finally, we applied similar transformations to those used in the augmentation step.

For the other models, we simply normalized the images in order to have zero mean and unitary variance.

C. Edge detection

Since the images with cacti are characterized mostly by straight lines, we extracted features based on the presence of straight lines: firstly we converted the images to grayscale, then we performed Canny edge detection to find edges in the images. We applied the Hough transform to highlight the lines and we computed the number of vertical, horizontal and oblique lines. We then analysed a batch of 32 images from the training set to check if there was some correlation between the number of lines found by the Hough transform and the presence of cacti. From Fig. 2 we can see that images with cacti are characterized by a large number of straight lines. We

will use this intuition in the implementation of the K-means algorithm.

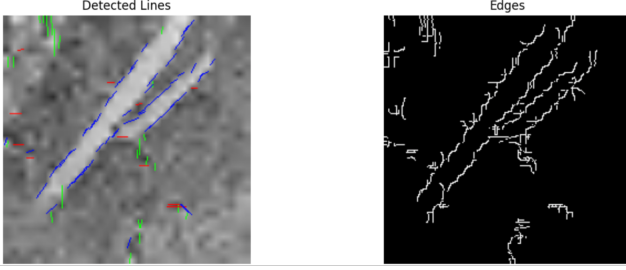


Fig. 2. Example of line detection

D. HOG Feature Extractor

HOG (Histogram of Oriented Gradients) [5] is a feature extraction technique used in image processing for object detection, particularly effective in capturing both the shape and local edge structure within an image. The main idea is that the appearance and shape of an object can be well described by the distribution of local intensity gradients, regardless of the absolute pixel intensity. Firstly, the image is divided into cells (8×8 pixels) where orientation histograms weighted by gradient magnitude are created. Subsequently, these cells are grouped into blocks (2×2 cells) that are normalized to improve the lighting invariance. The final HOG descriptor concatenates all normalized block histograms into a feature vector representing the structural patterns of the image. Finally, the features are used as input to the SVM and the Logistic regression classifiers.

E. Dataset Splitting

We split the dataset into a training, a validation and a test set using a 70/15/15 split for all models except for the K-means in which we performed a 70/30 split to hold out a test set.

IV. MODEL SELECTION

To address the task we tested several models, starting with traditional machine learning methods and then employing transfer learning with deep learning architectures:

- K-means clustering: a machine learning algorithm used to group data into k distinct clusters based on feature similarity. In this case it is easy to apply as we know the number of clusters. We will use as features the number of lines of different types in the image.
- SVM: a supervised learning algorithm used primarily for classification, that finds the optimal hyperplane that best separates data points of different classes in a high-dimensional space.
- Logistic Regression: a supervised learning algorithm used for binary classification that outputs the probability that an input belongs to a certain class using the logistic function.
- ResNet-50: a type of deep neural network architecture that addresses the vanishing gradient problem present in very deep networks by introducing residual connections.

- VGG-16: a type of deep neural network characterized by a very deep architecture composed mostly of small convolutional filters.

In order to evaluate the performance of the models we used the following metrics:

- F1 score: the harmonic mean of precision and recall, reflecting the trade-off between false positives and false negatives.
- Accuracy: measures the proportion of correct predictions out of all the predictions made.
- Binary cross-entropy: loss function used in binary classification tasks, it measures the difference between the predicted probability and the actual label.

The neural networks have been trained with a batch size of 32 and for a maximum of 50 epochs with an early stop condition defined for the loss on the validation set, in order to prevent overfitting on the training set once the loss on the validation set stops decreasing for a certain number of consecutive epochs. For all the experiments we set the seed to 42 for the reproducibility of results.

A. K-means

After having found that images containing a cactus have a high number of straight lines, we decided to compute the mean number of lines of various types for the two labels on all the training images: these values are then used as initial centroids for the K-means algorithm. Fig. 3 shows the mean values obtained. We ran the K-means algorithm with its default parameters, considering for each image a feature vector composed by the number of lines of different types. A label is assigned to the clusters based on the centroids, supposing that the centroid with higher mean values for the number of lines represents the positive label. After having separated the points in the two clusters, we compared the labels of the points in the two clusters to their ground truth labels, and we computed the values of accuracy and F1 score on the training and test sets: **Train accuracy: 0.655, Test accuracy: 0.649, Train F1 score: 0.536, Test F1 score: 0.530**. We can conclude that the algorithm obtains decent results that, however, can be vastly improved. For this reason, we chose this model as a baseline.

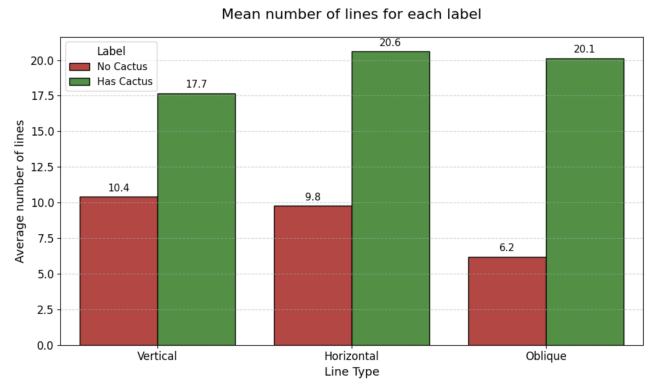


Fig. 3. Mean number of lines per label

B. SVM

We implemented a SVM, trying different types of kernel that allow to separate data in a high-dimensional space. We use as input the normalized images to which we applied the HOG feature extractor and we performed hyperparameter tuning with grid search on the hyperparameters in Table I, obtaining the following best configuration: **kernel: 'rbf'**, **gamma: 'scale'**, **C: 1.0**, and metrics' values on the validation set: **accuracy: 0.942, F1 score: 0.944**.

TABLE I
HYPERPARAMETERS SVM

Model	HyperParameter	Values
SVM	gamma	{0.01, 0.1, 0.5, 'scale'}
	kernel	{'rbf', 'poly', 'sigmoid'}
	C	{0.1, 1.0, 10.0}

We trained again with the best hyperparameters on the entire training set (including the validation set), obtaining the following results on the test set: **accuracy: 0.941, F1 score: 0.943**. We can conclude that this model obtains good results, which however can be further improved on.

C. Logistic Regression

Similarly as we did for the SVM, we used as input the normalized images to which we applied the HOG feature extractor and we performed hyperparameter tuning with grid search on the hyperparameters in Table II, obtaining the best configuration of hyperparameters: **C: 10.0, max_iter: 3000, tol: 1e-3**, and metrics' values on the validation set: **accuracy: 0.917, F1 score: 0.919, Cross-entropy loss: 0.211**. In this case we also included the value of the loss function used, which is the binary cross-entropy loss.

TABLE II
HYPERPARAMETERS LOGISTIC REGRESSION

Model	Hyperparameter	Values
Logistic Regression	max_iter	{1000, 2000, 3000}
	tol	{1e-4, 1e-3, 1e-2}
	C	{0.1, 1.0, 10.0}

We retrained with the best hyperparameters on the entire training set (including the validation set) and we obtained the following results on the test set: **Accuracy: 0.915, F1 score: 0.916, Log loss: 0.199**. We can see that the performance is slightly worse with respect to the SVM. This behaviour may be due to the fact that logistic regression is generally more sensitive to outliers than SVM. Nonetheless, this is still a good result.

D. ResNet-50

We used ResNet-50 as a feature extractor, since it was trained on ImageNet which is a dataset of natural images, so its features are appropriate for this task. In order to utilize this pretrained architecture, we froze all the layers of the ResNet (so that the weights are not updated during training) except for

the last fully connected layer, which we replaced with a custom head, made up of: a fully connected layer, a ReLU activation function, a Dropout layer with probability 0.3 and a final fully connected layer. This methodology allows to train only the custom head, reducing significantly the training times while still obtaining satisfactory results. We performed hyperparameter tuning with grid search on the hyperparameters in Table III, obtaining the following results for the best hyperparameters: **gamma: 0.1, step_size: 7, learning_rate: 0.001**, with the following metrics' values on the validation set: **Accuracy: 0.995, F1 score: 0.995, Cross-entropy Loss: 0.012**. The hyperparameters chosen are used in the Adam optimizer and the StepLR scheduler employed during the training process. Fig. 4 shows the trends of the F1 score on the training and validation sets. We did not report the plot of the accuracy because it has a similar trend to the F1 score.

TABLE III
HYPERPARAMETERS RESNET-50

Model	Hyperparameter	Values
ResNet-50	gamma	{0.01, 0.1, 0.5}
	step_size	{3, 5, 7, 10}
	learning_rate	{0.001, 0.01}

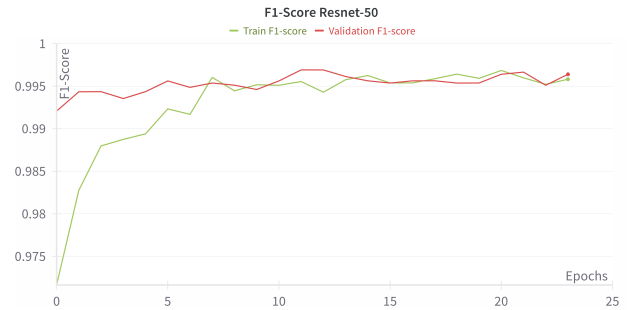


Fig. 4. F1 score trends of Resnet-50

We then retrained on the entire training set (including the validation set) for the maximum number of epochs reached during the hyperparameter tuning step and we obtained the following results on the test set: **Accuracy: 0.998, F1 score: 0.998, Cross-entropy Loss: 0.007**. These are really good results, highlighting the good capabilities of this model on the task of image classification.

E. VGG-16

Similarly as what we did for ResNet-50, we used VGG-16 as a feature extractor, since it was also trained on ImageNet. For this reason we froze all the layers of the model except for the last fully connected layer. The reason we decided to use a simple linear classification head in the case of VGG-16 is that the model already has a lot of parameters ($\sim 138M$), so we did not want to add unnecessary capacity to the model. On the other hand, in the case of ResNet-50, which has considerably less parameters ($\sim 25M$), we decided

it would be better to use a more expressive classification head. We performed hyperparameter tuning with grid search on the hyperparameters in Table IV, obtaining the following results for the best hyperparameters: **gamma : 0.01, step_size : 7, learning_rate: 0.01**, with the following metrics' values on the validation set: **accuracy: 0.986, F1 score: 0.986, Cross-entropy Loss: 0.113**. The hyperparameters chosen are used in the Adam optimizer and the StepLR scheduler employed during the training process. Fig. 5 shows the trends of the F1 score on the training and validation sets. We did not report the plot of the accuracy because it has a similar trend to the F1 score.

TABLE IV
HYPERPARAMETERS VGG-16

Model	Hyperparameter	Values
VGG-16	gamma	{0.01, 0.1, 0.5}
	step_size	{3, 5, 7, 10}
	learning_rate	{0.001, 0.01}

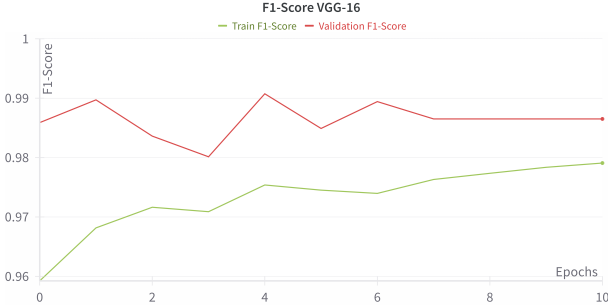


Fig. 5. F1 score trends of VGG-16

As we did for ResNet-50, we retrained on the training and validation sets together for the maximum number of epochs reached during the hyperparameter tuning step and we obtained the following results on the test set: **Accuracy: 0.989, F1 score: 0.989, Cross-entropy Loss: 0.108**. This demonstrates that VGG-16 has a similar performance to ResNet-50, despite having been trained for half the number of epochs.

V. UNLABELED TEST SET

To leverage the unlabeled test set provided we decided to use a Visual Question Answering (VQA) model: we chose a Vision-and-Language Transformer (ViLT), fine-tuned on VQAv2 [6]. This model, unlike traditional models that rely on convolutional neural networks or object detectors for image feature extraction, simplifies the process by treating images similarly to text, dividing the images into non-overlapping patches: this approach resembles how a text is tokenized into words. Each patch is then linearly projected into an embedding space, eliminating the need for complex visual feature extractors. We provided each image to the model with the prompt "Does this aerial image contain a cactus? Answer with yes or no". We then compared the labels assigned by this model to the

ones predicted by ResNet-50 and VGG-16. Fig. 6 shows how the models agree on the labels of the test set. We can notice that there is a strong discrepancy in agreement between the VQA and the CNN models. Therefore we decided to sample 200 images randomly from the test set and assign them labels manually, then we compared them to the labels predicted by the models. This experiment showed the following results: **VQA Accuracy: 0.535, ResNet-50 Accuracy: 0.885, VGG-16 Accuracy: 0.905**. We can therefore conclude that our models have a fairly good generalization capability.

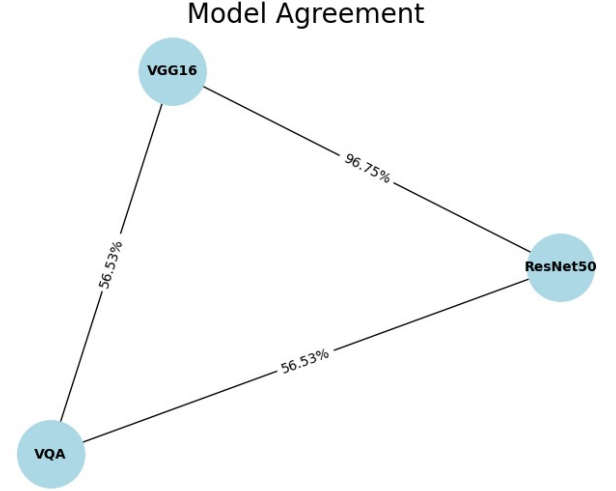


Fig. 6. Models' agreement on the unlabeled test set

VI. CONCLUSIONS AND FUTURE WORK

In this project we faced the problem of classifying aerial images in order to predict the presence of a cactus in the image. We performed several preprocessing steps on the data, including data augmentation, which was essential to balance the dataset, enhancing the models' generalization capabilities. To tackle this task we first implemented K-means clustering based on the number of lines of various types present in the image. After having established this baseline, we used other traditional machine learning approaches like SVM and logistic regression which obtained high performances. Finally we resorted to transfer learning with ResNet-50 and VGG-16 architectures, which reached really good results after having been trained for a low number of epochs. This is likely due to the fact that these models were trained on ImageNet. All the results of the different models are summarized in Table V.

TABLE V
RESULTS

Model	Train, Test Accuracy	Train, Test F1 score
K-means	0.655, 0.649	0.536, 0.530
SVM	0.967, 0.941	0.968, 0.943
Logistic Regressor	0.962, 0.915	0.961, 0.916
Resnet50	0.995, 0.998	0.995, 0.998
VGG	0.979, 0.989	0.979, 0.989

We also tried to use the unlabeled test set by employing a pretrained Visual Question Answering model, which however proved to be in disagreement with our best models, forcing us to manually label the test images to verify the generalization capability of our best models. As possible improvements, it would be useful to find some way of efficiently taking advantage of this test set.

REFERENCES

- [1] Juan Irving Vasquez, Vigia, 2016.
<https://jivg.org/research-projects/vigia/>
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition", 2016.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, "Very Deep Convolutional Networks for Large-Scale Image Recognition", 2014.
- [4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei, "Imagenet: A large-scale hierarchical image database", 2009.
- [5] Masahiro Terayama, Jungpil Shin, Won-Du Chang, "Object Detection using Histogram of Oriented Gradients", 2009.
- [6] ViLT model. <https://huggingface.co/dandelin/vilt-b32-finetuned-vqa>